

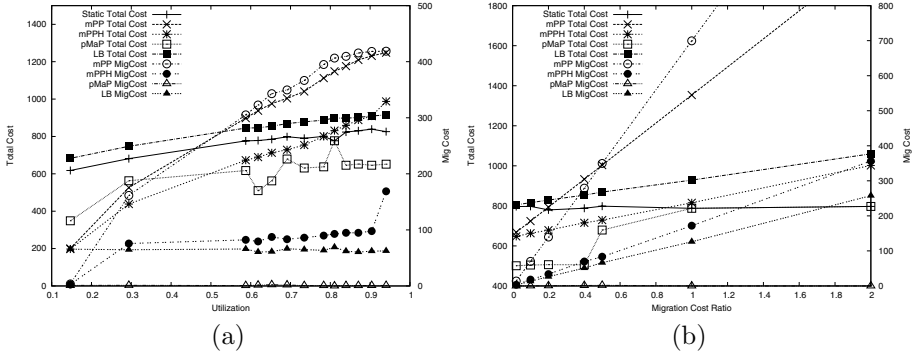


Fig. 8. Migration Cost and Overall Cost with (a) increasing overall utilization (b) change in MP (Migration to Power Cost) Ratio

migration cost and compare the power-minimizing algorithms with the migration cost aware *pMaP* algorithm as well (Fig. 8(a)). We observe that even though the power drawn by *pMaP* (difference of total cost and migration cost) is higher than *mPP* or *mPPH*, the total cost of *pMaP* is the least amongst all competing algorithms. This establishes the importance of taking both the migration and power cost into account, while coming up with a new placement. We also observe that *mPP* pays a very high migration cost at higher utilization, and underperforms even the *LB* (Load – balanced) and *Static* placements at very high loads. On the other hand, *mPPH* incurs very low migration cost (of the same order as load-balanced) and as a result, has a total cost very close to the best performing algorithm *pMaP*. This is a direct result of the fact that *mPPH* takes the previous placement into account while computing a new placement, and tries to minimize the difference between the two placements. Hence, even though *mPPH* does not compromise on saving power as opposed to *pMaP*, which can prefer a placement with high power and low migration cost, the migration costs incurred by *mPPH* are not significant. This establishes *mPPH* as a good alternative to *pMaP*, because of its relative simplicity.

The *MP* ratio (migration to power cost ratio) varies from one enterprise to another and depends on factors such as energy cost and SLA revenue models. We next investigate the various algorithms at different *MP* ratio. We observed (Fig. 8(b)) that *MP* ratio directly affects the overall performance of *mPP*. Hence, *mPP* is the best performing algorithm (approaching *mPPH* and *pMaP*) at low migration costs and underperforms even the power-unaware algorithms at very high migration cost. This underscores the importance of taking migration cost into account, along with the power cost for dynamic placement. On the other hand, *pMaP* takes the increased migration cost factor into account by cutting down migrations. Hence, a high *MP* ratio does not affect its performance.

In our next set of experiments, we investigated the scalability of the algorithms along with their ability to deal with fragmentation (large VM sizes). Hence, we increased the number of servers from 4 to 16. We observed that *pMaP* and